

WEALTHAI

Technical Design & Developer Reference

Full-Stack Indian Fintech Wealth Management Platform

Version	1.0 — Fully Updated
Date	27 May 2026
Status	Ready for Development
Audience	Development Team

1. WealthAI — Project Overview

WealthAI is a comprehensive Indian fintech and wealth management platform. It enables users to track, manage, and analyse their entire financial portfolio across multiple asset classes — all in one place. Supports multiple user profiles under one login, multiple broker accounts per profile, and covers everything from government savings schemes to live algo trading.

1.1 Core Modules

Module	Description	DB Schema
Authentication	Accounts, login, sessions, multi-profile support, broker linking	auth
Deposits	FD, RD, PPF, EPF, SSY, NPS, NSC, SCSS, KVP — tracking & interest	deposits
Bonds	Govt, corporate, tax-free bond holdings and transactions	bond
Mutual Funds	Schemes, NAV history, holdings, SIP/STP/SWP, dividends	mutual_fund
India Market	Equity & F&O portfolio, orders, market data, FII/DII, VIX, MMI	india_market
Algo Engine	Strategy builder, deployments, live/paper/backtest trading	algo

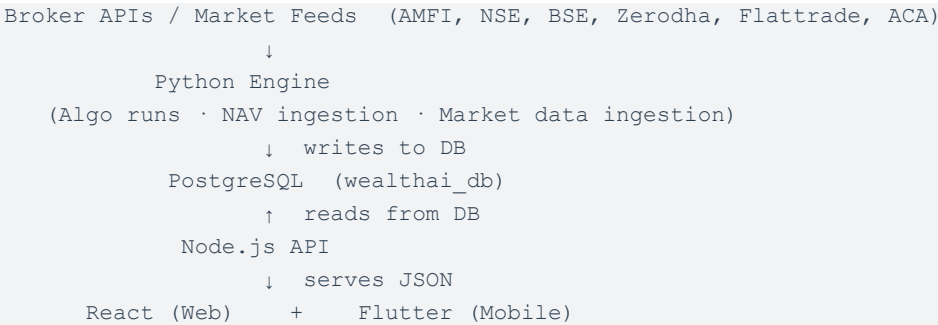
1.2 Key Principles

- Multi-profile: One login account manages portfolios for self, spouse, children, parents, clients
- Multi-broker: Each user profile links multiple demat/trading accounts
- Separation of concerns: Python handles computation and algo; Node.js handles CRUD and serving
- Indian market focus: INR, NSE/BSE/NFO exchanges, AMFI NAV feeds, EPFO integrations
- Soft deletes everywhere: `is_deleted` flag — never hard delete user data
- UUID primary keys throughout: `gen_random_uuid()` for all tables

2. Technology Stack

Layer	Technology	Purpose
Web Frontend	React	Dashboard, portfolio views, strategy builder, charts
Mobile Frontend	Flutter	iOS & Android — mirrors all web features
CRUD API	Node.js + Express + TypeScript	REST API — all data fetch and write operations
Data & Algo API	Python	Algo engine, NAV ingestion, market data, heavy computation
Database	PostgreSQL	Primary data store — 6 schemas, 30+ tables

2.1 Data Flow For Algo Engine



2.2 Node vs Python — Responsibility Split

API	Handles
Node.js (TypeScript)	All CRUD · Fetch and serve data to frontend · Read algo output · Auth & sessions · Scheduler (triggers Python)
Python	AMFI NAV fetch & parse · Algo strategy & Automation execution · Market data ingestion · XIRR & return calculations · Contract note parsing

3. Application Folder Structure (Top Level)

The project is organised into four top-level directories providing clean separation between backend, frontend, database and documentation.

```
WealthAI/                                     ← Root of the entire project
|
├── backend/                                  ← All server-side code
│   ├── node-api/                            ← Node.js REST API (TypeScript)
│   └── python-api/                         ← Python (algo engine, data ingestion)
├── frontend/                               ← All client-side code
│   ├── web-app/                            ← React web application
│   └── mob-app/                            ← Flutter mobile application
├── database/                               ← All DB related files
│   ├── migrations/                        ← SQL schema files (001 → 006)
│   └── seeds/                             ← Seed data (instruments, MF schemes etc.)
└── documents/                             ← Project documentation
    └── WealthAI_Design_Document.docx
```

4. Frontend — React Web Application

React web application serving the dashboard, portfolio views, strategy builder and all user-facing screens. Communicates with Node.js API.

4.1 Folder Structure

```
frontend/web-app/
├── public/
├── src/
│   ├── pages/
│   │   ├── auth/
│   │   ├── dashboard/
│   │   ├── deposits/
│   │   ├── bonds/
│   │   ├── mutual-funds/
│   │   ├── equity/
│   │   ├── fno/
│   │   └── algo/
│   ├── components/
│   │   ├── common/
│   │   └── charts/
│   ├── api/
│   │   ├── auth.api.ts
│   │   ├── deposits.api.ts
│   │   ├── bonds.api.ts
│   │   ├── mf.api.ts
│   │   ├── market.api.ts
│   │   └── algo.api.ts
│   ├── store/
│   ├── utils/
│   ├── App.tsx
└── package.json
```

← One folder per module
← Login, Register
← Portfolio summary
← FD, PPF, EPF etc.
← Strategy, Deployments, Runs
← Reusable UI components
← Button, Card, Table, Badge etc.
← Portfolio charts, P&L graphs
← Axios API layer (one file per module)
← State management
← Formatters, date helpers, INR format

4.2 UI Summary View — Field Reference

i Maximum 8 fields per module in summary view. Gain/Loss: green if positive, red if negative. All monetary values in INR (₹) format.

Deposits

#	Field	DB Source	Sample Value
1	Account Name	account_name	My SBI FD
2	Type	account_type	FD / PPF / EPF (badge)
3	Institution	institution_name	SBI
4	Invested Amount	principal_amount	₹1,00,000
5	Current Value	current_value	₹1,14,500
6	Gain / Loss	current_value – principal_amount	+ ₹14,500

#	Field	DB Source	Sample Value
7	Interest Rate	interest_rate	7.10% p.a.
8	Maturity Date	maturity_date	12 Mar 2027

Bonds

#	Field	DB Source	Sample Value
1	Bond Name	bond_master.bond_name	7.26% GOI 2032
2	Type	bond_master.bond_type	Govt / Corporate (badge)
3	Invested Amount	bond_holdings.invested_amount	₹1,00,000
4	Current Value	quantity × current_price (computed)	₹1,06,000
5	Gain / Loss	computed	+ ₹6,000
6	Coupon Rate	bond_master.coupon_rate	7.26%
7	Maturity Date	bond_master.maturity_date	15 Jan 2032
8	Status	bond_holdings.status	Active

Mutual Funds

#	Field	DB Source	Sample Value
1	Scheme Name	mf_schemes.scheme_name	Mirae Asset Large Cap Fund
2	Fund House	mf_schemes.fund_house	Mirae Asset
3	Category	mf_schemes.category + sub_category	Equity — Large Cap
4	Invested Amount	mf_holdings.invested_amount	₹12,700
5	Current Value	units × current_nav (computed)	₹14,400
6	Gain / Loss	computed	+ ₹1,700
7	Gain %	computed	+ 13.4%
8	1D Change	mf_nav_history.day_change_percent	+ 0.45%

Equity Holdings

#	Field	DB Source	Sample Value
1	Company Name	instruments.name	Reliance Industries
2	Symbol	instruments.symbol	RELIANCE
3	Quantity	equity_holdings.quantity	50
4	Avg Buy Price	equity_holdings.avg_buy_price	₹2,450
5	Current Price	equity_holdings.current_price	₹2,780
6	Invested Amount	quantity × avg_buy_price (computed)	₹1,22,500

#	Field	DB Source	Sample Value
7	Current Value	quantity × current_price (computed)	₹1,39,000
8	Unrealised P&L	equity_holdings.unrealised_pnl	+ ₹16,500

F&O Positions

#	Field	DB Source	Sample Value
1	Symbol	instruments.symbol	NIFTY
2	Type + Strike	instruments.option_type + strike_price	CE 24500
3	Expiry	instruments.expiry_date	29 May 2026
4	Buy / Sell	fno_positions.transaction_type	BUY
5	Lots	fno_positions.lots	2
6	Avg Price	fno_positions.avg_price	₹320
7	Current Price	fno_positions.current_price	₹410
8	Unrealised P&L	fno_positions.unrealised_pnl	+ ₹9,000

Algo — Strategies

#	Field	DB Source	Sample Value
1	Strategy Name	algo.strategies.strategy_name	Iron Condor Weekly
2	Capital	algo.strategies.capital	₹5,00,000
3	Status	algo.strategies.enabled	Active / Disabled (badge)
4	Visibility	algo.strategies.is_public	Public / Private (badge)

Algo — Deployments (with Latest Run inline)

i Fetch latest deployment_run (ORDER BY run_date DESC LIMIT 1) per deployment. Shaded rows are run-level fields.

#	Field	DB Source	Sample Value
1	Strategy Name	algo.strategies.strategy_name	Iron Condor Weekly
2	Broker	algo.brokers.broker_name	Zerodha
3	Multiplier	algo.deployments.multiplier	2x
4	Mode	algo.deployments.deployment_mode	Live / Paper (badge)
5	Capital	algo.deployments.capital_allocated	₹2,00,000
6	Total P&L	algo.deployments.total_pnl	+ ₹14,500
7	Status	algo.deployments.status	Active / Paused (badge)
—	Latest Run P&L	deployment_runs.pnl (latest row)	+ ₹3,200
—	Latest Run Date	deployment_runs.run_date (latest row)	17 May 2026

#	Field	DB Source	Sample Value
—	Run Selector	Dropdown — run_date newest → oldest	17 May / 16 May...

5. Frontend — Flutter Mobile Application

Flutter mobile application for iOS and Android. Mirrors all web features. Communicates with same Node.js API.

5.1 Folder Structure

```
frontend/mob-app/  
├── lib/  
│   ├── screens/                ← One screen per module  
│   │   ├── auth/  
│   │   ├── dashboard/  
│   │   ├── deposits/  
│   │   ├── bonds/  
│   │   ├── mutual_funds/  
│   │   ├── equity/  
│   │   └── algo/  
│   ├── widgets/                ← Reusable Flutter widgets  
│   ├── services/               ← API service layer (Dio)  
│   │   ├── auth_service.dart  
│   │   ├── deposits_service.dart  
│   │   ├── mf_service.dart  
│   │   └── algo_service.dart  
│   ├── models/                 ← Data models / DTOs  
│   └── main.dart  
└── pubspec.yaml
```

6. Backend — Node.js API (TypeScript)

REST API for all CRUD operations and data serving to React and Flutter. Built with Express.js, TypeScript, and raw pg driver for PostgreSQL.

6.1 Folder Structure

```
backend/node-api/  
├── src/  
│   ├── index.ts                ← App entry point  
│   └── modules/                ← One folder per module  
│       ├── auth/  
│       │   ├── controllers/  
│       │   │   ├── auth.controller.ts  
│       │   │   └── profile.controller.ts  
│       │   ├── routes/  
│       │   │   └── auth.routes.ts  
│       │   └── services/  
│       │       ├── auth.services.ts  
│       │       └── profile.services.ts  
│       ├── bond/  
│       │   ├── controllers/bond.controller.ts  
│       │   ├── routes/bond.routes.ts  
│       │   └── services/bond.services.ts  
│       ├── deposits/  
│       │   ├── controllers/deposits.controller.ts  
│       │   ├── routes/deposits.routes.ts  
│       │   └── services/deposits.services.ts  
│       ├── mutual_fund/  
│       │   ├── controllers/  
│       │   │   ├── mf.crud.controller.ts  
│       │   │   ├── mf.nav.controller.ts  
│       │   │   └── mf.sip.controller.ts  
│       │   ├── routes/mf.routes.ts  
│       │   └── services/  
│       │       ├── mf.crud.services.ts  
│       │       ├── mf.nav.services.ts  
│       │       └── mf.sip.services.ts  
│       ├── algo_engine/  
│       │   ├── controllers/  
│       │   │   ├── strategy.controller.ts  
│       │   │   ├── deployment.controller.ts  
│       │   │   └── runs.controller.ts  
│       │   ├── routes/strategy.routes.ts  
│       │   └── services/  
│       │       ├── strategy.services.ts  
│       │       ├── deployment.services.ts  
│       │       └── runs.services.ts  
│       └── india_market/  
│           ├── controllers/  
│           │   ├── equity.controller.ts  
│           │   ├── fno.controller.ts  
│           │   ├── instruments.controller.ts  
│           │   └── marketdata.controller.ts
```

```

├── routes/market.routes.ts
├── services/
│   ├── equity.services.ts
│   ├── fno.services.ts
│   ├── instruments.services.ts
│   └── marketdata.services.ts
├── portfolio/
│   ├── controllers/portfolio.controller.ts
│   ├── routes/portfolio.routes.ts
│   └── services/portfolio.services.ts
├── shared/
│   ├── dbconnection.ts      ← PostgreSQL pool
│   ├── logger.ts           ← Winston logger
│   ├── pythonRunner.ts     ← Spawn Python scripts
│   ├── response.ts         ← sendSuccess / sendError helpers
│   ├── scheduler.ts        ← Cron job manager
│   ├── types.ts            ← Shared TypeScript interfaces
│   └── middleware/
│       ├── auth.middleware.ts ← JWT verify
│       └── errorHandler.ts  ← Global error handler
├── package.json
├── tsconfig.json
├── .env.example
├── .gitignore
└── README.md

```

6.2 File Naming Convention

File	Purpose
*.controller.ts	Request/response only — calls services, returns JSON. No SQL here.
*.services.ts	All DB queries + business logic. No res/req objects.
*.routes.ts	Express router — maps HTTP methods and paths to controllers.

6.3 API Endpoints

Authentication — /api/v1/auth

Method	Endpoint	Description
POST	/auth/register	Register new account
POST	/auth/login	Login — returns access_token + refresh_token
POST	/auth/logout	Invalidate session
POST	/auth/refresh	Get new access_token using refresh_token
GET	/auth/me	Get current account details
POST	/auth/users	Add user profile to account
GET	/auth/users	Get all user profiles
PUT	/auth/users/:id	Update user profile

Method	Endpoint	Description
POST	/auth/brokers	Link broker account to user
GET	/auth/brokers	Get all linked broker accounts

Deposits — /api/v1/fd

Method	Endpoint	Description
POST	/fd/add	Add deposit account (FD/RD/PPF/EPF etc.)
GET	/fd/all	Get all deposit accounts for user
GET	/fd/:id	Get single deposit with transactions
PUT	/fd/update/:id	Update deposit account
DELETE	/fd/delete/:id	Soft delete deposit account
POST	/fd/:id/transaction	Add transaction to deposit
GET	/fd/:id/transactions	Get all transactions for deposit

Bonds — /api/v1/bond

Method	Endpoint	Description
POST	/bond/add	Add bond holding
GET	/bond/all	Get all bond holdings
GET	/bond/:id	Get single holding with transactions
PUT	/bond/update/:id	Update bond holding
DELETE	/bond/delete/:id	Soft delete bond holding
POST	/bond/:id/transaction	Add buy/sell/interest transaction
GET	/bond/master	Get bond master catalog

Mutual Funds — /api/v1/mf

Method	Endpoint	Description
GET	/mf/schemes	Search/list MF schemes from master
GET	/mf/schemes/:id	Get scheme details with metadata
GET	/mf/nav/:scheme_id	Get NAV history for scheme
GET	/mf/holdings	Get all MF holdings for user
POST	/mf/holdings/add	Add new MF holding/folio
POST	/mf/transaction	Record buy/sell/switch transaction
GET	/mf/sip/all	Get all SIP/STP/SWP plans
POST	/mf/sip/add	Create new systematic plan

Method	Endpoint	Description
PUT	/mf/sip/:id	Update or cancel systematic plan
POST	/mf/fetch-nav	Trigger Python: download NAVAll.txt from AMFI
POST	/mf/parse-nav	Trigger Python: parse and load to DB

Algo Engine — /api/v1/algo

Method	Endpoint	Description
POST	/algo/add	Create new strategy
GET	/algo/all	Get all strategies for user
GET	/algo/:id	Get strategy with sets and positions
PUT	/algo/update/:id	Update strategy config
DELETE	/algo/delete/:id	Soft delete strategy
POST	/algo/:id/deploy	Deploy strategy (live/paper/backtest)
GET	/algo/deployments/all	Get all deployments for user
PUT	/algo/deployments/:id/status	Pause/stop/resume deployment
GET	/algo/runs/:deployment_id	Get all daily runs for deployment
GET	/algo/runs/:run_id/trades	Get trade log and orderbook for run
GET	/algo/runs/:run_id/positions	Get positions for run

India Market — /api/v1/market

Method	Endpoint	Description
GET	/market/instruments	Search instruments by symbol/exchange/type
GET	/market/equity/holdings	Get equity holdings for user
GET	/market/equity/transactions	Get equity trade history
GET	/market/fno/positions	Get open F&O positions
GET	/market/fno/transactions	Get F&O trade history
GET	/market/ohlcv	Get historical OHLCV data
GET	/market/fii-dii	Get FII/DII cashflow data
GET	/market/options-chain	Get options chain for symbol+expiry
GET	/market/mmi	Get latest Market Mood Index
POST	/market/historical	Trigger Python: fetch OHLCV for ticker

Portfolio — /api/v1/portfolio

Method	Endpoint	Description
--------	----------	-------------

Method	Endpoint	Description
GET	/portfolio/summary	Aggregated portfolio value across all modules
GET	/portfolio/allocation	Asset allocation breakdown by module
GET	/portfolio/pnl	Overall realised + unrealised P&L

7. Backend — Python API

Data ingestion, algo execution engine and complex analysis layer. Called by Node.js API via `pythonRunner.ts` for all heavy computation tasks.

7.1 Folder Structure

```

backend/python-api/
├── app/                                ← All application code
│   ├── __init__.py
│   └── main.py                        ← Entry point
│   └── shared/                        ← Shared utilities
│       ├── __init__.py
│       ├── bootstrap.py              ← Fix import paths
│       ├── constants.py              ← All paths & config
│       ├── db.py                     ← PostgreSQL connection
│       ├── logger.py                 ← Centralised logger
│       └── response.py                ← Response helpers
│   └── modules/
│       ├── __init__.py
│       ├── algo_engine/              ← Algo trading engine
│           ├── __init__.py
│           ├── start_engine.py        ← Engine orchestrator
│           ├── strategy_runner.py     ← Thread wrapper
│           └── strategy_loader.py     ← JSON → Strategy object
│       ├── executor/                 ← FIXED — core execution
│           ├── __init__.py
│           ├── strategy_executor.py
│           ├── set_executor.py
│           └── position_executor.py
│       ├── engine/                   ← EXPANDING — capabilities
│           ├── __init__.py
│           ├── entry_checker.py
│           ├── exit_checker.py
│           ├── risk_checker.py
│           ├── adjustment_handler.py
│           ├── indicators.py
│           └── candle_helper.py
│       └── services/                  ← Service managers
│           ├── __init__.py
│           ├── service_hub.py
│           ├── log_manager.py
│           ├── status_manager.py
│           ├── order_manager.py
│           ├── position_manager.py
│           ├── trade_manager.py
│           ├── tsl_manager.py
│           └── performance_manager.py

```

```

└─ context_manager.py

└─ brokers/          ← Broker integrations
    └─ __init__.py
    └─ base.py
    └─ broker_manager.py
    └─ simulated.py
    └─ paper.py
    └─ flattrade/
    └─ finvasia/
    └─ aca/
    └─ zerodha/

└─ datafeed/         ← Market data providers
    └─ __init__.py
    └─ market_data_provider.py

└─ db/               ← Database layer
    └─ __init__.py
    └─ db_manager.py
    └─ db_pool.py
    └─ db_helper.py
    └─ db_interface.py
    └─ json_db.py

└─ utils/
    └─ __init__.py
    └─ constants.py
    └─ logger.py
    └─ utility.py

└─ mutual_fund/      ← MF data ingestion
    └─ __init__.py
    └─ fetch_nav_txt.py
    └─ fetch_from_wayback.py
    └─ parse_nav_txt.py
    └─ load_nav_to_db.py
    └─ sync_scheme_master.py

└─ market_data/      ← Market data ingestion
    └─ __init__.py
    └─ ohlcv_fetcher.py
    └─ fii_dii_fetcher.py
    └─ mmi_fetcher.py
    └─ vix_fetcher.py
    └─ options_chain_fetcher.py
    └─ corporate_actions_fetcher.py
    └─ importers/     ← Contract note parsers
        └─ __init__.py
        └─ base_importer.py
        └─ zerodha/
            └─ contract_note.py
        └─ flattrade/
            └─ contract_note.py
        └─ finvasia/

```



```

├── contract_note.py
├── deposits/
│   ├── __init__.py
│   ├── interest_calculator.py
│   ├── maturity_forecaster.py
│   └── scheme_rates_sync.py
├── portfolio/
│   ├── __init__.py
│   ├── xirr_calculator.py
│   ├── pnl_aggregator.py
│   └── allocation_analyser.py
├── data/
│   └── mfnave/                ← NAV .txt and .csv files
├── logs/
├── requirements.txt
├── .env.example
└── .gitignore

```

7.2 Daily Scheduled Jobs

i Located at: `node-api/src/shared/scheduler.ts` — uses `node-cron`. Invokes Python via `pythonRunner.ts`

Job	Schedule IST	Script
NAV Fetch	Daily 10:00 PM	<code>mutual_fund/fetch_nav_txt.py</code>
NAV Parse & Load	Daily 10:30 PM	<code>mutual_fund/parse_nav_txt.py</code>
OHLCV Fetch	Weekdays 4:00 PM	<code>market_data/ohlcw_fetcher.py</code>
FII / DII Data	Weekdays 5:00 PM	<code>market_data/fii_dii_fetcher.py</code>
MMI Score	Weekdays 6:00 PM	<code>market_data/mmi_fetcher.py</code>
Algo Engine	Manual trigger (live later)	<code>algo_engine/start_engine.py</code>

8. Database — PostgreSQL

Database: PostgreSQL (wealthai_db). Six isolated schemas. UUIDs as primary keys. TIMESTAMPTZ for all timestamps. Soft deletes via is_deleted flag.

8.1 Migrations

Located at: database/migrations/. Run in order 001 → 006 to set up full schema.

File	Schema	Tables	Covers
001_db_creation.sql	auth	6	accounts, sessions, users, brokers, settings, notifications
002_bond.sql	bond	3	bond_master, bond_holdings, bond_transactions
003_mutual_fund.sql	mutual_fund	8	mf_benchmarks, mf_schemes, mf_nav_history, mf_scheme_metadata, mf_dividends, mf_systematic_plans, mf_holdings, mf_transactions
004_algo_engine.sql	algo	10	brokers, strategies, strategy_sets, strategy_positions, deployments, deployment_runs, trade_logs, trade_orderbook, trade_positions, strategy_subscriptions
005_india_market.sql	india_market	15	instruments, instrument_metadata, orders, equity_holdings, equity_transactions, fno_positions, fno_transactions, dividends, corporate_actions, historical_ohlc, fii_dii_cashflow, fii_derivatives, mmi, options_data, market_volatility_index
006_deposits.sql	deposits	3	account_holdings, account_transactions, account_metadata

8.2 Seeds

Located at: database/seeds/. Run after migrations. Populates reference data the app needs to function.

File	Populates
001_instruments.sql	NSE/BSE equity and F&O instruments master
002_mf_schemes.sql	AMFI mutual fund scheme master with ISINs
003_mf_benchmarks.sql	Benchmark indices — NIFTY 50 TRI, SENSEX TRI etc.
004_deposit_rates.sql	Current government interest rates for PPF, SSY, SCSS, NSC etc.
005_lot_sizes.sql	F&O lot sizes for NIFTY, BANKNIFTY, stocks

8.3 Schema Table Reference

auth — User & Identity (6 tables)

Table	Purpose
auth.logins	Login credentials — username, email, password hash, role

Table	Purpose
auth.login_sessions	Device and IP session tracking with refresh tokens
auth.users	User profiles per account — self, spouse, child, parent, client
auth.user_brokers	Linked demat/trading accounts per user
auth.user_settings	Per-user preferences — theme, currency, timezone
auth.user_notifications	In-app notification store per user

bond — Fixed Income (3 tables)

Table	Purpose
bond.bond_master	ISIN-level bond catalog — govt, corporate, tax-free, G-Sec, SDL, T-Bills
bond.bond_holdings	User bond positions — platform, quantity, avg price, status
bond.bond_transactions	Buy/sell/interest credit/maturity events per holding

mutual_fund — Mutual Funds (8 tables)

Table	Purpose
mutual_fund.mf_benchmarks	Benchmark index master — NIFTY 50 TRI, SENSEX TRI etc.
mutual_fund.mf_schemes	AMFI scheme master — scheme code, ISIN, fund house, plan type
mutual_fund.mf_nav_history	Daily NAV per scheme — one row per scheme per date
mutual_fund.mf_scheme_metadata	AUM, expense ratio, Sharpe, Alpha, Beta, star rating
mutual_fund.mf_dividends	Dividend payout history per scheme
mutual_fund.mf_systematic_plans	SIP/STP/SWP plans per user — amount, frequency, mandate
mutual_fund.mf_holdings	User holdings — folio number, units, avg NAV, invested amount
mutual_fund.mf_transactions	Buy/sell/switch/dividend reinvest transactions

algo — Algo Engine (10 tables)

Table	Purpose
algo.brokers	Broker API credentials per user for algo execution
algo.strategies	Top-level strategy blueprint — capital, entry/exit conditions, risk rules
algo.strategy_sets	Groups of positions within a strategy — segment, exchange config
algo.strategy_positions	Individual trade legs — option type, strike selection, order type
algo.deployments	Live/Paper/Backtest deployment — user + strategy + broker combo
algo.deployment_runs	One execution record per day — P&L, capital, start/end time
algo.trade_logs	Engine decisions and events timestamped during a run
algo.trade_orderbook	Every order placed — broker and exchange order IDs

Table	Purpose
algo.trade_positions	Entry/exit lifecycle per position — price, time, P&L
algo.strategy_subscriptions	Marketplace — who follows whose strategy

india_market — Stock Market (15 tables)

Table	Purpose
india_market.instruments	Master for all equities, F&O, ETFs, indices
india_market.instrument_metadata	Index membership — Nifty50/100, Midcap150, sector, industry
india_market.orders	All orders — manual, algo, copy trade
india_market.equity_holdings	Current delivery holdings — qty, avg buy price, unrealised P&L
india_market.equity_transactions	Confirmed buy/sell trades — delivery and intraday
india_market.fno_positions	Open F&O positions — futures and options
india_market.fno_transactions	Entry/exit/expiry F&O trades with realised P&L
india_market.dividends	Dividend receipts per stock
india_market.corporate_actions	Splits, bonus, rights, mergers, demergers, buybacks
india_market.historical_ohlc	OHLCV price history — all timeframes in one table
india_market.fii_dii_cashflow	Daily FII and DII institutional buy/sell/net flow
india_market.fii_derivatives	FII positions in F&O segment
india_market.options_data	Options chain — OI, IV, LTP per strike per expiry per day
india_market.mmi	Market Mood Index — daily sentiment score
india_market.market_volatility_index	India VIX and other volatility indices — daily OHLC

deposits — Savings Schemes (3 tables)

Table	Purpose
deposits.account_holdings	One row per deposit per user — FD/RD/PPF/EPF/SSY/NPS/NSC/SCSS/KVP
deposits.account_transactions	All financial events — deposits, withdrawals, interest, EPF contributions
deposits.account_metadata	Govt-announced interest rate history per scheme — used for forecasting

9. Deployment Strategy

9.1 Environments

Environment	Purpose	Notes
Development	Local developer machines	All services run locally — PostgreSQL, Node, Python, React dev server
Staging	Testing before production	Mirrors production — used for QA and integration testing
Production	Live user-facing system	Cloud hosted — all services containerised

9.2 Recommended Stack

Component	Service	Notes
PostgreSQL DB	AWS RDS / Neon	Managed DB — automated backups, scaling
Node.js API	AWS EC2 / Render	Run with PM2 for process management
Python Engine	AWS EC2 / Render	Cron-based daily runs — keep separate from Node
React Web	Vercel / Netlify	Static build — fast CDN delivery
Flutter Mobile	Google Play + Apple App Store	Standard mobile deployment

9.3 Environment Variables

```
# PostgreSQL
DB_HOST=localhost
DB_PORT=5432
DB_NAME=wealthai_db
DB_USER=postgres
DB_PASSWORD=your_password

# JWT
JWT_SECRET=your_jwt_secret
JWT_EXPIRES_IN=15m
REFRESH_TOKEN_SECRET=your_refresh_secret

# Python
PYTHON_PATH=python3
PYTHON_API_DIR=../python-api

# CORS
ALLOWED_ORIGINS=http://localhost:3001,http://localhost:5173
```

10. Conventions, Roadmap & Notes

10.1 Naming Conventions

Layer	Convention	Example
DB tables & columns	snake_case	bond_holdings, avg_purchase_price
Node files	module.type.ts	mf.controller.ts, bond.services.ts
API endpoints	kebab-case	/india-market/historical
Python files	snake_case	fetch_nav_txt.py, parse_nav_txt.py
React components	PascalCase	BondHoldingCard.tsx, MFSSchemeList.tsx
Flutter widgets	PascalCase	DepositCard(), PortfolioSummary()

10.2 Database Rules

- All primary keys: UUID using `gen_random_uuid()`
- All timestamps: `TIMESTAMPZ` — never plain `TIMESTAMP`
- Soft deletes: set `is_deleted = true` — never hard delete user data
- Schema prefix mandatory in all queries — e.g. `SELECT * FROM auth.users`
- All foreign keys to user profiles reference `auth.users(id)`

10.3 Node API Rules

- Controllers handle request/response only — no SQL ever in controllers
- Services handle all DB queries and business logic
- All routes prefixed with `/api/v1/`
- JWT middleware applied to all routes except `/auth/register` and `/auth/login`
- Always use `sendSuccess()` / `sendError()` from `shared/response.ts`

10.4 Python Algo Engine Rules

- Always run from `algo_engine/` root: `cd app/modules/algo_engine && python start_engine.py`
- ServiceHub is thread-local — each deployment run gets isolated services
- DB writes use parameterised queries — no f-string SQL
- `deployment_runs` table is updated by engine — `deployments` table is updated by user action only